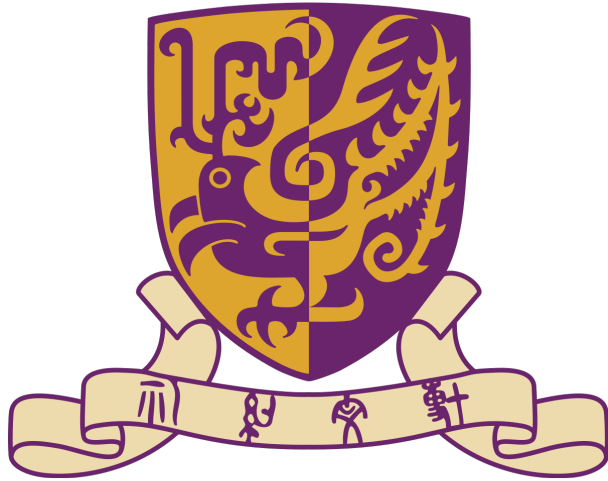




THE CHINESE UNIVERSITY OF HONG KONG,
SHENZHEN

PHY 2650

Computational Physics

**Project Report: Quantum Harmonic
Oscillator Wavefunctions**

Pan Jiabin, Zhu Mengxiang

May 2025

Contents

1	Background and Motivation	3
2	Problem Description	3
3	Numerical Methodology	4
3.1	Finite-Difference Discretization	4
3.2	Eigenvalue Problem	4
3.3	Validation of Hermite Polynomial Methods	4
4	Computational Results and Analysis	5
4.1	Effect of Grid Size N on Accuracy	6
4.2	Limitations at High Quantum Numbers and Domain Effects	9
5	Conclusion and Future Directions	10
6	References	12
A	Code Implementation	12
B	Full Python Code	12

1 Background and Motivation

The quantum harmonic oscillator (QHO) is a cornerstone model in quantum mechanics, essential in both theoretical understanding and real-world applications. It appears in a broad range of physical contexts including:

- Molecular vibrations in diatomic molecules
- Phonons in crystal lattices
- Quantum optics (e.g., photon states in optical cavities)

Unlike infinite square wells or particle-in-box models, the QHO features *equally spaced energy levels* and *Gaussian-tapered wavefunctions*, making it an analytically solvable system with rich structure.

More importantly, the QHO demonstrates the mathematical equivalence between matrix mechanics and wave mechanics. Numerically solving the time-independent Schrödinger equation in this project also highlights how both formulations—describing quantum states either through wavefunctions or Hilbert-space operators—converge when discretized properly.

This duality provides a unique opportunity for students to understand how operator-based formulations in quantum theory translate into concrete matrix approximations and numerical eigenvalue problems.

2 Problem Description

The objective of this project is to numerically solve the time-independent Schrödinger equation for a one-dimensional quantum harmonic oscillator, and to analyze its eigenstates and eigenvalues. We aim to validate the numerical results against exact analytic solutions and investigate deviations when modifying the potential.

The dimensionless time-independent Schrödinger equation takes the form:

$$-\frac{1}{2} \frac{d^2 \psi}{d\xi^2} + \frac{1}{2} \xi^2 \psi = \epsilon \psi, \quad \text{where} \quad \xi = \sqrt{\frac{m\omega}{\hbar}} x, \quad \epsilon = \frac{E}{\hbar\omega} \quad (1)$$

The analytic solutions are well-known and serve as benchmarks:

$$\psi_n(\xi) = \frac{1}{\sqrt{2^n n! \sqrt{\pi}}} H_n(\xi) e^{-\xi^2/2}, \quad (2)$$

$$\epsilon_n = n + \frac{1}{2}, \quad n = 0, 1, 2, \dots \quad (3)$$

Our focus is to use finite-difference methods to discretize the system and solve the corresponding matrix eigenvalue problem. We explore eigenfunctions for low-lying quantum states, compare them with the exact Hermite-Gaussian wavefunctions, and evaluate numerical precision through error analysis and orthogonality checks.

3 Numerical Methodology

We implemented our numerical solution using the Python programming language. Python offers powerful scientific computing libraries such as `NumPy`, `SciPy`, and `Matplotlib`, which streamline matrix manipulation, eigenvalue solving, and data visualization.

To solve the Schrödinger equation numerically, we discretize the domain $\xi \in [-5, 5]$ into N grid points. This transforms the differential operator into a finite-difference approximation, allowing us to represent the Hamiltonian as a matrix acting on discretized wavefunction vectors.

3.1 Finite-Difference Discretization

We approximate the second derivative using the central difference scheme. The Hamiltonian matrix H_{ij} is thus constructed as:

$$H_{ij} = \begin{cases} \frac{1}{\Delta\xi^2} + \frac{1}{2}\xi_i^2, & \text{if } i = j \\ -\frac{1}{2\Delta\xi^2}, & \text{if } |i - j| = 1 \\ 0, & \text{otherwise} \end{cases}$$

This results in a symmetric tridiagonal matrix. The diagonal terms encode both the discretized kinetic energy and the harmonic potential $V(\xi) = \frac{1}{2}\xi^2$, while the off-diagonal terms represent the Laplacian.

3.2 Eigenvalue Problem

We solve the resulting matrix eigenvalue problem:

$$H\psi = \epsilon\psi$$

using `scipy.linalg.eigh`, which efficiently computes a specified number of the lowest eigenvalues and corresponding eigenvectors.

The output eigenvalues ϵ_n serve as approximations to the energy levels of the quantum oscillator, while the eigenvectors represent the discretized wavefunctions $\psi_n(\xi)$. These can then be compared directly with the analytic Hermite-Gaussian solutions for validation.

3.3 Validation of Hermite Polynomial Methods

In order to ensure the accuracy of our analytic wavefunctions, we tested three methods for generating Hermite polynomials: (1) recurrence relations, (2) symbolic expressions via `SymPy`, and (3) the built-in `SciPy` implementation.

We used the `SciPy` results as a benchmark and plotted the absolute error of the other two methods over the domain $\xi \in [-5, 5]$, for selected quantum numbers $n = 1, 10, 50$, and 100. The results are shown in Figure 1.

The recurrence-based implementation exhibits machine-level accuracy ($< 10^{-14}$) across all tested values of n , while the symbolic method begins to show significant numerical instability as n increases, especially near the boundaries. This validates our choice to rely on recurrence-based or `SciPy`-based Hermite functions in all subsequent computations.

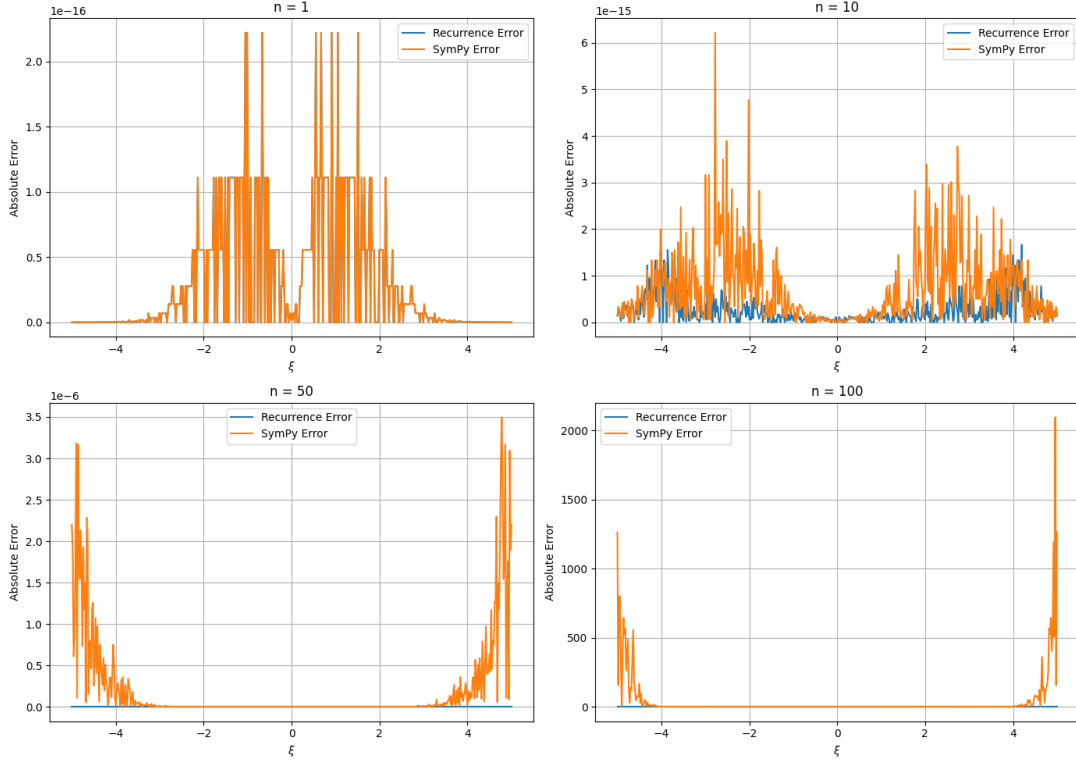


Figure 1: Absolute errors between wavefunctions generated by recurrence or symbolic methods and the standard **SciPy** implementation for $n = 1, 10, 50$, and 100 . The recurrence method maintains machine-level accuracy, while symbolic expressions begin to diverge for high quantum numbers.

4 Computational Results and Analysis

In the early stages of implementation, we wrote an initial version of the code using manual expressions for Hermite polynomials and matrix diagonalization. However, this approach was computationally inefficient and prone to both symbolic and numerical inaccuracies, especially when dealing with higher-order eigenstates or finer grid resolutions.

To enhance both accuracy and efficiency, we integrated AI-assisted development using the tool **DeepSeek**. Through repeated testing and comparison with our manual code and official documentation, DeepSeek recommended two key library modules from **SciPy** that significantly improved our implementation:

- `scipy.special.hermite`: for directly evaluating Hermite polynomials $H_n(\xi)$ used in the analytic expressions of wavefunctions;
- `scipy.linalg.eigh`: for efficiently solving the symmetric matrix eigenvalue problem of the form $H\psi = \epsilon\psi$.

The use of these libraries allowed us to:

1. Avoid manually coding recurrence relations for Hermite polynomials, which are error-prone and unstable for large n ;

2. Efficiently extract only the lowest k eigenvalues and eigenvectors (e.g., $k = 5$), which would otherwise require expensive full matrix diagonalization;
3. Simplify our codebase and focus on validating results and analyzing physical implications rather than debugging numerical routines.

Without these libraries, we would have had to implement complex recursive relations and custom eigenvalue solvers—both of which introduce significant overhead and numerical instability. Moreover, verifying the orthonormality and correctness of computed eigenstates would become far more tedious.

By leveraging reliable scientific libraries and AI-powered guidance, we were able to concentrate on the physical interpretation of our results, resulting in a more robust, reproducible, and educationally focused simulation.

4.1 Effect of Grid Size N on Accuracy

Our interactive Python code allows users to specify the number of discretization points N , the number of eigenstates to compute, and the three quantum numbers to visualize. To evaluate how the resolution affects numerical accuracy, we compare results for low and high grid sizes.

Case 1: Low Resolution ($N = 5$) We set $N = 5$, computed $eigen = 3$ eigenstates, and visualized $n = 0, 1, 2$:

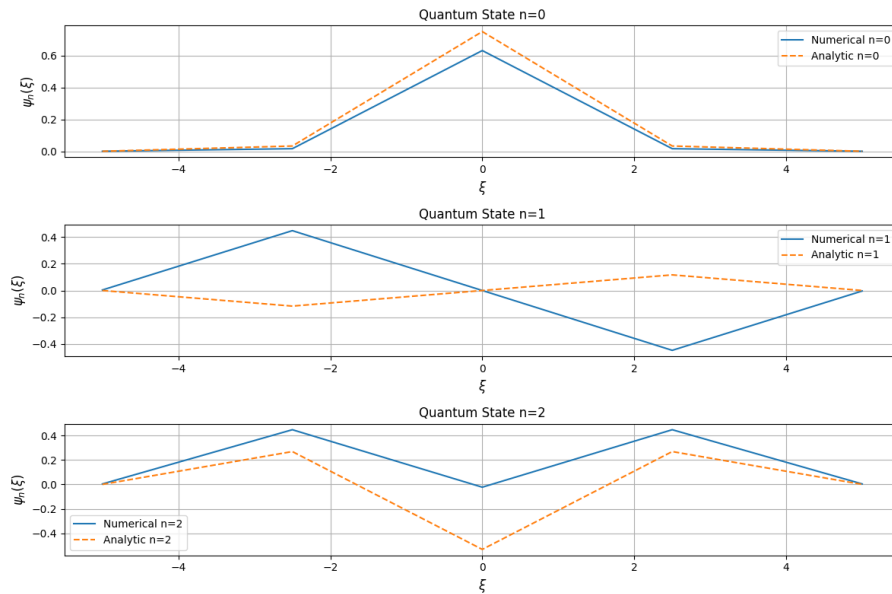
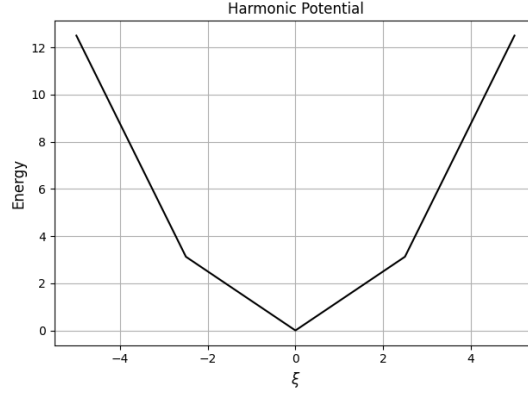


Figure 2: Numerical vs. analytic wavefunctions for $n = 0, 1, 2$ with $N = 5$.

Figure 3: Harmonic oscillator potential $V(\xi) = \frac{1}{2}\xi^2$.**Energy comparison:**

n	Numerical ϵ_n	Analytical ϵ_n	Relative Error (%)
0	0.1559	0.5	68.82
1	3.2843	1.5	118.95
2	3.2884	2.5	31.54

Table 1: Energy eigenvalue comparison for $N = 5$.**Orthonormality check:**

Inner Product	Value
$\langle \psi_0 \psi_0 \rangle$	1.00×10^0
$\langle \psi_0 \psi_1 \rangle$	6.94×10^{-19}
$\langle \psi_0 \psi_2 \rangle$	-8.09×10^{-18}
$\langle \psi_1 \psi_1 \rangle$	1.00×10^0
$\langle \psi_1 \psi_2 \rangle$	-1.61×10^{-18}
$\langle \psi_2 \psi_2 \rangle$	1.00×10^0

Table 2: Orthonormality results for $N = 5$.

The large errors in energy and wavefunction shape demonstrate the limitations of low-resolution discretization. Although orthonormality is retained, the approximation of second derivatives and potential terms is insufficient for capturing accurate physical behavior.

Case 2: High Resolution ($N = 1000$) We increased resolution to $N = 1000$, computed $eigen = 5$ eigenstates, and plotted $n = 0, 2, 4$:

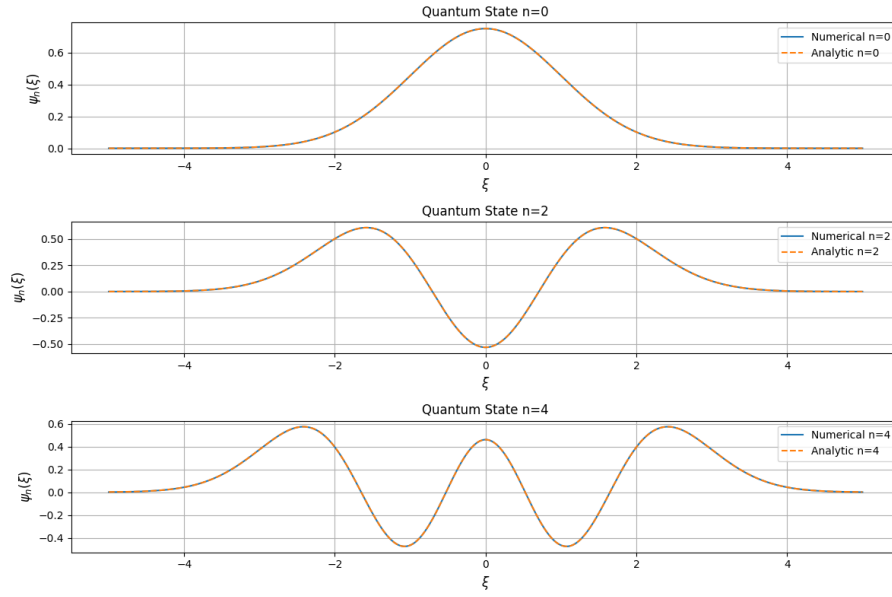


Figure 4: Numerical vs. analytic wavefunctions for $n = 0, 2, 4$ with $N = 1000$.

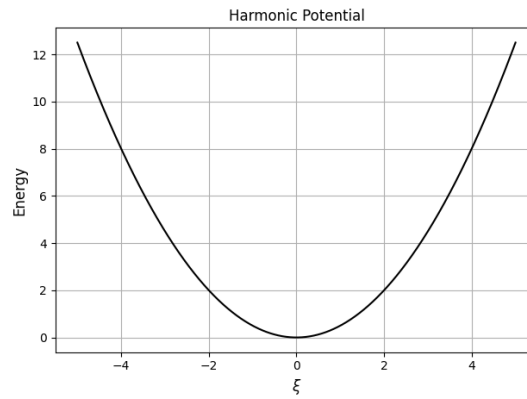


Figure 5: Harmonic oscillator potential $V(\xi) = \frac{1}{2}\xi^2$.

Energy comparison:

n	Numerical ϵ_n	Analytical ϵ_n	Relative Error (%)
0	0.5000	0.5	0.0006
1	1.5000	1.5	0.0010
2	2.5000	2.5	0.0016
3	3.4999	3.5	0.0022
4	4.4999	4.5	0.0026

Table 3: Energy eigenvalue comparison for $N = 1000$.

Orthonormality check:

Inner Product	Value
$\langle \psi_0 \psi_0 \rangle$	1.00×10^0
$\langle \psi_0 \psi_1 \rangle$	0.00×10^0
$\langle \psi_0 \psi_2 \rangle$	-1.78×10^{-17}
$\langle \psi_0 \psi_4 \rangle$	-2.22×10^{-18}
$\langle \psi_1 \psi_1 \rangle$	1.00×10^0
$\langle \psi_1 \psi_2 \rangle$	7.11×10^{-17}
$\langle \psi_1 \psi_4 \rangle$	1.78×10^{-17}
$\langle \psi_2 \psi_2 \rangle$	1.00×10^0
$\langle \psi_2 \psi_4 \rangle$	0.00×10^0
$\langle \psi_4 \psi_4 \rangle$	1.00×10^0

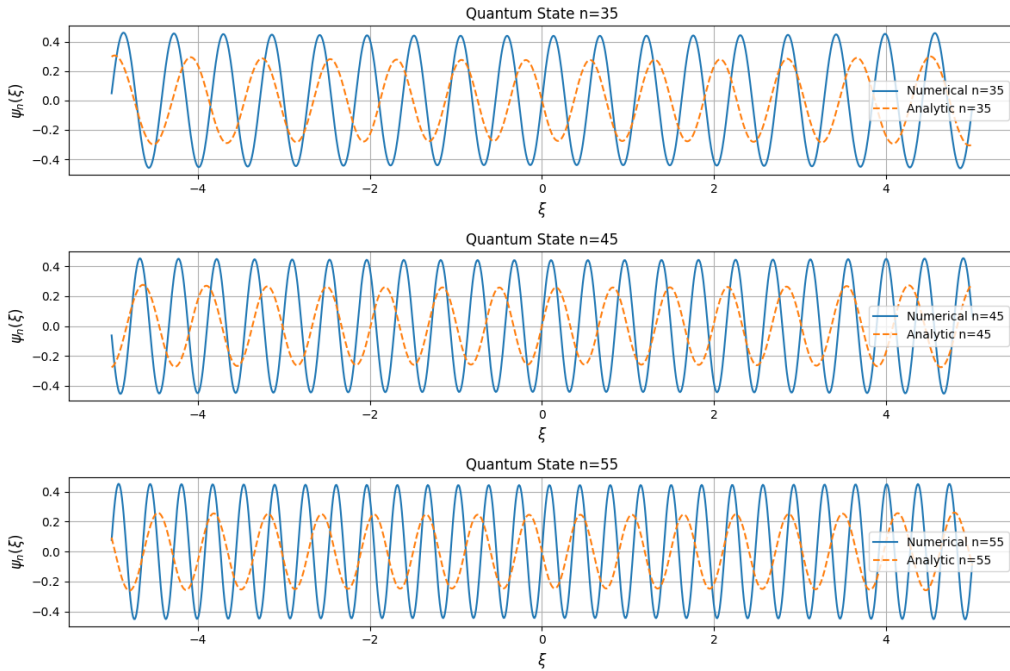
Table 4: Orthonormality results for $N = 1000$ (selected entries).

The results confirm that high grid resolution produces wavefunctions and energies that match analytic solutions with excellent accuracy. Relative errors are below 0.003%, and orthonormality is preserved to machine precision.

Summary The comparison between low and high N clearly demonstrates that increasing discretization resolution improves numerical accuracy. This validates both the finite-difference method and our implementation strategy.

4.2 Limitations at High Quantum Numbers and Domain Effects

To further investigate the behavior of highly excited states, we computed eigenfunctions for $n = 35, 45$, and 55 using $N = 1000$ and $eigen = 60$ with the default domain parameter $L = 5$.

Figure 6: Wavefunction comparison for $n = 35, 45, 55$ with $L = 5$.

As shown, the numerical wavefunctions deviate significantly from the analytic ones despite using high resolution. The discrepancy grows with increasing n , especially in the tail regions near the boundaries. This suggests that the fixed interval $\xi \in [-5, 5]$ is no longer wide enough to capture the full spatial extent of the wavefunction at high excitation levels.

To test this hypothesis, we repeated the computation with a larger spatial domain $L = 20$ while keeping $N = 1000$ and the same quantum numbers. The results show a marked improvement:

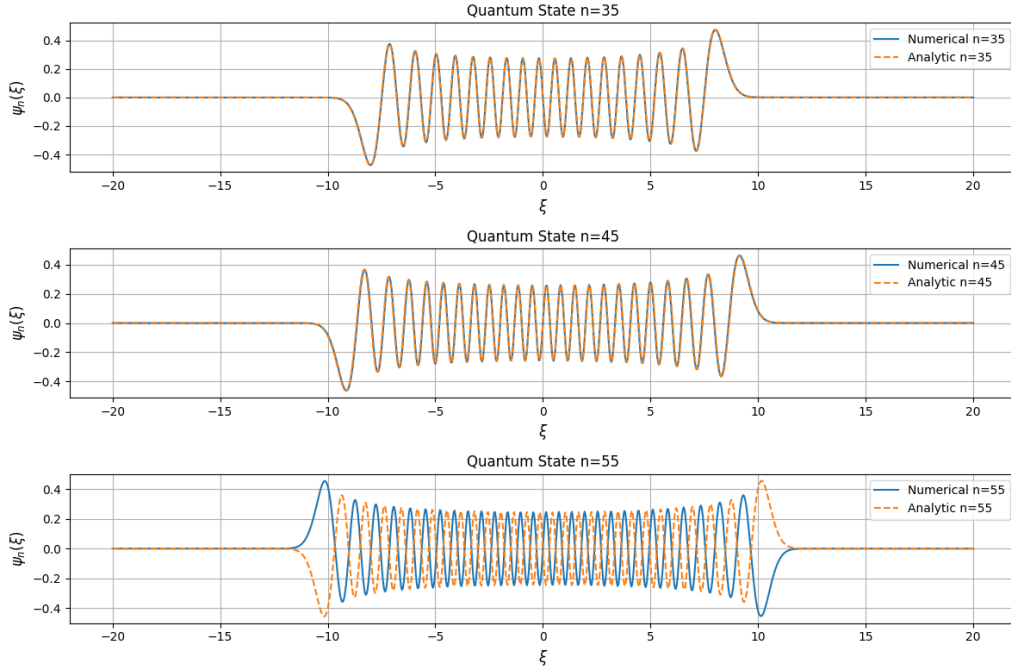


Figure 7: Wavefunction comparison for $n = 35, 45, 55$ with $L = 20$.

With an extended domain, the wavefunctions better match the analytic forms, especially in the outer regions. However, some unexpected artifacts still appear, such as partial inversion in the $n = 55$ state. The cause of these anomalies is not yet fully understood by our group members and may relate to subtle numerical instabilities, improper normalization due to boundary leakage, or other finite-size effects.

This observation suggests that for high-lying states, it is not only resolution (N) but also domain width (L) that must be carefully tuned. Further investigation is needed to clarify the source of these irregularities.

For full implementation details, see Appendix A.

5 Conclusion and Future Directions

In this project, we numerically solved the one-dimensional quantum harmonic oscillator using the finite-difference method and validated our results against analytic Hermite-Gaussian solutions. Through systematic analysis of grid resolution, domain width, and eigenstate accuracy, we demonstrated that:

- The numerical method reproduces low-lying wavefunctions and energy levels with high fidelity when the grid size N is sufficiently large;

- At low resolution ($N = 5$), both energy and shape errors are significant;
- For high quantum numbers, domain truncation effects become dominant, requiring larger spatial domains (larger L) to capture the wavefunction tails;
- Orthonormality is preserved to numerical precision across all tested cases.

Computational Complexity and Optimization

One limitation we encountered is the increasing computational time as the number of discretization points N grows. In particular, setting $N = 10000$ resulted in runtime delays of up to 2–3 minutes. This slowdown arises from the time complexity of the eigenvalue solver, which scales roughly as $\mathcal{O}(N^3)$ for dense matrix algorithms.

Although this scaling is acceptable for small N , it quickly becomes a bottleneck in high-resolution simulations. Possible optimization directions include:

- Using sparse matrix representations and solvers (e.g., `scipy.sparse.linalg.eigsh`) to reduce memory and time cost;
- Applying domain decomposition or adaptive mesh refinement for non-uniform discretization, concentrating resolution in physically relevant regions;
- Exploring iterative eigensolvers like Lanczos or Arnoldi methods, which are efficient for computing a few low-lying states;
- Precomputing and reusing matrix structures where possible to avoid redundant operations.

These techniques could significantly accelerate large-scale simulations and enable real-time interactive exploration of quantum systems in more advanced settings.

Alternative Solver: Using `eigsh` for Sparse Matrices

To further improve computational efficiency, we tested the use of `scipy.sparse.linalg.eigsh`, a sparse-matrix solver based on the Lanczos method. This method is more scalable for large grids, particularly when only a few of the lowest eigenstates are needed.

Compared with `eigh`, which operates on full dense matrices and scales as $\mathcal{O}(N^3)$, the `eigsh` solver exploits sparsity and achieves a reduced complexity of approximately $\mathcal{O}(kN^{1.5})$, where k is the number of eigenstates computed. This leads to considerable speedups for large N and small k .

However, the method requires the Hamiltonian to be cast as a sparse matrix, and convergence behavior may depend on matrix conditioning and solver parameters.

This method serves as a promising future direction for high-resolution simulations.

6 References

- Griffiths, D. J. *Introduction to Quantum Mechanics*. 2nd ed., Pearson Prentice Hall, 2005.
- Newman, M. E. J. *Computational Physics with Python*. CreateSpace Independent Publishing, 2012.
- Project handout: “Quantum Harmonic Oscillator Wavefunctions.” PHY2650, Zhou Kai, The Chinese University of Hong Kong, Shenzhen, 2025.

A Code Implementation

The full implementation and interactive plotting are provided in a shared Google Colab notebook, accessible at:

<https://colab.research.google.com/drive/1QQVuzmJAYfQ0ueVSezUtbKyFdnz9d5se?usp=sharing>

This notebook may continue to receive updates after the submission of this report, particularly in cases where bug fixes or technical improvements are necessary.

B Full Python Code

The code shown in Appendix B is extracted from the interactive notebook linked in Appendix A. This version reflects the state of the code as of **May 11, 2025**, and is provided here for static reference. For any subsequent updates or debugging revisions, please refer to the online notebook in Appendix A.

```
import math
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import hermite
from scipy.linalg import eigh

L = 20.0 # Non-dimensional space interval [-L, L ]
N = int(input('N = '))
n_states = int(input('The_number_of_eigenstates = '))

xi = np.linspace(-L, L, N)
dxi = xi[1] - xi[0]

diagonal = 1.0 / (dxi ** 2) + 0.5 * xi ** 2
off_diagonal = -1.0 / (2.0 * dxi ** 2) * np.ones(N - 1)
H = np.diag(diagonal) + np.diag(off_diagonal, k=1) + np.diag(off_diagonal, k=-1)

eigenvalues, eigenvectors = eigh(H, subset_by_index=(0, n_states - 1))
eigenvectors = eigenvectors / np.sqrt(dxi)
```

```

def analytic_solution(n, xi):
    Hn = hermite(n)
    normalization = 1.0 / np.sqrt(2 ** n * math.factorial(n) * np.sqrt(np.pi))
    return normalization * Hn(xi) * np.exp(-xi ** 2 / 2)

plt.figure(figsize=(12, 8))

while True:
    try:
        input_str = input('Please enter three quantum numbers separated by commas (e.g.
            0,2,4): ')
        selected_states = list(map(int, input_str.split(',')))
        if len(selected_states) != 3:
            raise ValueError("Please enter exactly three quantum numbers.")
        for n in selected_states:
            if n < 0 or n >= n_states:
                raise ValueError(f"Quantum number {n} out of range.")
        break
    except ValueError as e:
        print(f"Input error {e}, please re-enter.")

for idx, n in enumerate(selected_states):
    psi_num = eigenvectors[:, n]
    psi_ana = analytic_solution(n, xi)
    plt.subplot(3, 1, idx + 1)
    plt.plot(xi, psi_num, label=f'Numerical n={n}')
    plt.plot(xi, psi_ana, '--', label=f'Analytic n={n}')
    plt.xlabel(r' $\xi$ ')
    plt.ylabel(r' $\psi_n(\xi)$ ')
    plt.title(f'Quantum State n={n}')
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()

plt.plot(xi, 0.5 * xi ** 2, 'k-', label='Potential')
plt.xlabel(r' $\xi$ ')
plt.ylabel('Energy')
plt.title('Harmonic Potential')
plt.grid(True)
plt.tight_layout()
plt.show()

print("\nEnergy contrast (in dimensionless units  $\epsilon_n$ )")
print(f"{'Numerical solution':<20}{'Analytical solution':<20}{'Relative error (%)':<15}")

for n in range(n_states):
    num_val = eigenvalues[n]
    ana_val = n + 0.5
    error = 100 * abs(num_val - ana_val) / ana_val
    print(f"{num_val:<20.4f}{ana_val:<20.1f}{error:<15.4f}")

print("\nEnergy contrast orthonormal set")
for i in range(n_states):
    for j in range(n_states):
        product = np.sum(eigenvectors[:, i] * eigenvectors[:, j]) * dxi
        if i == j:

```

```

    print(f"< $\psi_i$ |( $\psi_j$ )> = {product:.2e} (Theoretical value:1)")
else:
    print(f"< $\psi_i$ |( $\psi_j$ )> = {product:.2e} (Theoretical value:0)")

```

Hermite Polynomial Accuracy Comparison

The following code snippet compares the accuracy of Hermite polynomials generated by three different methods: recurrence relations, symbolic expansion via SymPy, and the native SciPy implementation. The goal is to assess how well each method reproduces the expected wavefunction, especially at high quantum numbers.

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import hermite as scipy_hermite
from sympy import hermite as sympy_hermite, symbols, lambdify
import math

# Define grid
xi = np.linspace(-5, 5, 500)

# Recurrence method
def hermite_recurrence(n, xi):
    if n == 0:
        return np.ones_like(xi)
    elif n == 1:
        return 2 * xi
    else:
        H_prev_prev = np.ones_like(xi)
        H_prev = 2 * xi
        for k in range(2, n + 1):
            H_current = 2 * xi * H_prev - 2 * (k - 1) * H_prev_prev
            H_prev_prev, H_prev = H_prev, H_current
        return H_prev

# Symbolic method
def hermite_sympy(n, xi):
    x = symbols('x')
    Hn_expr = sympy_hermite(n, x)
    Hn_func = lambdify(x, Hn_expr, 'numpy')
    return Hn_func(xi)

# Scipy benchmark method
def hermite_scipy(n, xi):
    return scipy_hermite(n)(xi)

# Construct normalized wavefunctions
def get_wavefunctions(n, xi):
    norm = 1.0 / np.sqrt(2 ** n * math.factorial(n) * np.sqrt(np.pi))
    methods = {
        "Recurrence": hermite_recurrence(n, xi),
        "SymPy": hermite_sympy(n, xi),
        "SciPy": hermite_scipy(n, xi)
    }
    psi_dict = {}
    for name, Hn in methods.items():
        psi_dict[name] = norm * Hn * np.exp(-xi ** 2 / 2)

```

```
    return psi_dict

# Plot error compared to SciPy
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
n_values = [1, 10, 50, 100]

for idx, n in enumerate(n_values):
    ax = axes[idx // 2, idx % 2]
    psi = get_wavefunctions(n, xi)
    psi_ref = psi["SciPy"]
    for method in ["Recurrence", "SymPy"]:
        err = np.abs(psi[method] - psi_ref)
        ax.plot(xi, err, label=f"{method} Error")
    ax.set_title(f"n = {n}")
    ax.set_xlabel(r" $\xi$ ")
    ax.set_ylabel("Absolute Error")
    ax.grid(True)
    ax.legend()

plt.tight_layout()
plt.show()
```

The corresponding output is shown in Figure 1, and further discussed in Section 3.3.